Image Segmentation Based on Visual Prompt

Mai Duc Minh Huy
Faculty of Information and Technology
VNUHCM University of Science
Ho Chi Minh City, Vietnam
minhhuymaiduc@gmail.com

Chung Tin Dat
Faculty of Information and Technology
VNUHCM University of Science
Ho Chi Minh City, Vietnam
chungdatk5@gmail.com

***Abstract*—Image segmentation, a cornerstone of computer vision, involves partitioning an image into meaningful segments, a task fundamental to high-level image understanding. This paper presents a comprehensive survey of the evolution of image segmentation techniques. We trace the trajectory from early, pre-deep learning methods that relied on handcrafted features and mathematical models, to the revolutionary impact of the deep learning era, marked by architectures like Fully Convolutional Networks (FCN), U-Net, and Mask R-CNN. We then delve into the modern landscape, which is increasingly dominated by Transformer-based architectures and the paradigm of foundation models. A significant focus is placed on the Segment Anything Model (SAM), a recent breakthrough that introduced the concept of promptable, zero-shot segmentation, aiming to create a single, generalist model for a vast array of segmentation tasks. This survey analyzes the motivations, problem formulations, and core contributions of these pivotal works, providing a structured overview of the field's progress and highlighting the paradigm shift towards generalized, prompt-driven segmentation.**

***Index Terms*—Image Segmentation, Deep Learning, Foundation Models, Segment Anything (SAM), Computer Vision**

I. Introduction

A. General Background

Image segmentation is a fundamental task in computer vision that involves partitioning a digital image into multiple segments or sets of pixels, often corresponding to different objects or parts of objects. The goal is to simplify or change the representation of an image into something that is more meaningful and easier to analyze. The history of this field can be broadly divided into three distinct eras.

The Pre-Deep Learning Era (c. 1970s–2000s): Before the widespread adoption of deep learning, segmentation methods were primarily based on handcrafted features and mathematical models. These techniques required significant domain knowledge and focused on pixel-level properties such as intensity, color, texture, and edges. Prominent methods from this period include:

- **Canny Edge Detection:** Proposed by John Canny in 1986, this method became a standard for detecting a wide range of edges in images through a multi-stage algorithm [1].
- **Active Contour Models (Snakes):** Introduced by Kass et al. in 1988, these deformable models were particularly effective for segmenting objects with irregular shapes, finding extensive use in medical imaging [2].
- **K-Means Clustering:** This unsupervised algorithm, formalized by MacQueen in 1967, partitions pixels into clusters based on feature similarity, serving as a foundational clustering-based segmentation technique [3].

These methods, while foundational, relied on heuristics and low-level features, often struggling with complex scenes and semantic understanding.

The Deep Learning Era (c. 2010s): The advent of deep learning, particularly Convolutional Neural Networks (CNNs), revolutionized image segmentation. These models could automatically learn hierarchical features from data, eliminating the need for manual feature engineering. Key milestones include:

- **Fully Convolutional Networks (FCN):** Long et al. (2015) introduced FCNs, which replaced fully connected layers in CNNs with convolutional layers, enabling end-to-end training for dense, pixel-wise prediction. The use of skip connections allowed the combination of deep, semantic features with shallow, appearance features [4].
- **U-Net:** Proposed by Ronneberger et al. (2015), U-Net features a symmetric encoder-decoder architecture with extensive skip connections. This design proved highly effective for biomedical image segmentation, achieving remarkable performance with very few training images [5].
- **Mask R-CNN:** He et al. (2017) extended the Faster R-CNN object detection framework by adding a parallel branch for predicting segmentation masks. Mask R-CNN became a dominant framework for instance segmentation and a versatile backbone for many downstream tasks [6].

Modern Research Directions: Current research continues to build upon these deep learning foundations, exploring new architectures and learning paradigms for greater accuracy, efficiency, and generalization.

- **Transformer-Based Architectures:** Vision Transformers (ViTs) and self-attention mechanisms have been adapted for segmentation, leading to models like TransUNet (2021) that combine the strengths of Transformers and U-Net [7].
- **Foundation Models:** A recent paradigm shift involves building large-scale, pre-trained models that can be adapted to various tasks with minimal fine-tuning. The Segment Anything Model (SAM) from Meta AI (2023) exemplifies this, offering prompt-based, zero-shot seg-

mentation for images [8]. Its successor, SAM 2 (2024), extends these capabilities to video [9].
- **Self-Supervised Learning:** Techniques like contrastive learning (e.g., SimCLR) are being explored to reduce reliance on large labeled datasets, enabling models to learn powerful representations from unlabeled data.

B. Motivation

The drive to advance image segmentation stems from both scientific inquiry and practical necessity.

Scientifically, segmentation addresses a fundamental question in perception: while image classification tells us *what* is in an image, segmentation answers *where* it is, down to the pixel level. This granular understanding is crucial for emulating the human ability to parse a scene into distinct objects and regions, forming the basis for true visual comprehension.

Practically, segmentation is an enabling technology for a multitude of real-world applications:

- **Autonomous Driving:** Segmenting lanes, pedestrians, vehicles, and traffic signs is critical for safe navigation.
- **Medical Imaging:** Precise segmentation of tumors, organs, and other anatomical structures is vital for diagnosis, treatment planning, and surgical guidance.
- **Real-World Data Complexity:** Segmentation provides a robust way to handle complex scenes with occluded or overlapping objects.
- **Downstream Vision Tasks:** It serves as a foundational step for numerous other tasks, including object detection, tracking, and detailed semantic scene understanding. Without accurate segmentation, the performance of these downstream applications would be severely limited.

The motivation behind the **Segment Anything Model (SAM)** represents a culmination of these goals. As stated by its creators, “In this work, our goal is to build a foundation model for image segmentation” [8]. The aim was not to create another specialized model but a generalist one. They sought “to develop a promptable model and pre-train it on a broad dataset using a task that enables powerful generalization. With this model, we aim to solve a range of downstream segmentation problems on new data distributions using prompt engineering” [8]. This ambition marks a shift towards creating a single, powerful tool that can handle any segmentation request on any image, democratizing the technology and accelerating research and application development.

C. Problem Statement

Image segmentation is the task of grouping pixels that belong to a common semantic object and separating them from other objects and the background. A ”semantic object” refers to an entity that is meaningful and recognizable within a given context, such as a car, a person, a tree, or even amorphous regions like ”sky” or ”road”.

The core principle of segmentation is guided by two criteria: pixels within the same group should be as similar as possible, while pixels belonging to different groups should be as distinct as possible. This ”similarity” can be defined by various attributes, including color, intensity, texture, or learned semantic features. In essence, the task simulates the human cognitive ability to transform raw, unstructured visual data into a structured representation of distinct entities.

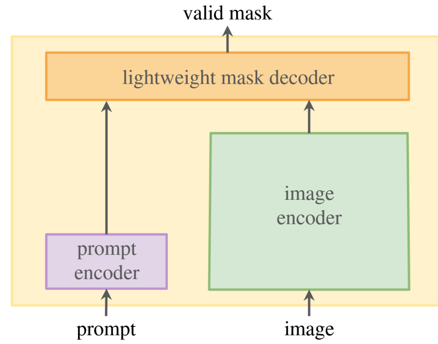

Fig. 1. A general framework for a promptable segmentation model. An image encoder computes a feature embedding for the input image. A prompt encoder embeds the user’s prompt (e.g., points, boxes). A lightweight mask decoder combines these embeddings to predict segmentation masks in real-time.

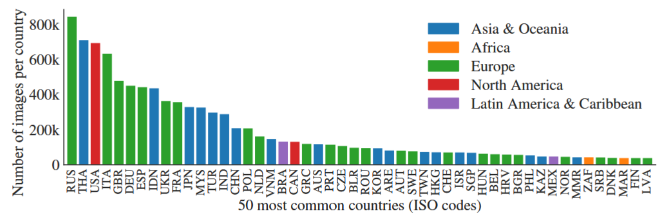

Fig. 2. Estimated geographic distribution of SA-1B images. The map shows that most countries are represented with over 1000 images, making it one of the most geographically diverse datasets for computer vision. (Source: [8])

For a modern promptable segmentation system like SAM, the problem can be formally defined by its inputs and outputs.

- **Input:** An image and a ”visual prompt” that specifies the object of interest. This prompt can take various forms, such as a point (or multiple points) on the object, a bounding box enclosing it, or even a rough mask.
- **Output:** One or more segmentation masks for the object(s) indicated by the prompt, along with a confidence score for each mask. The ability to output multiple masks is crucial for handling ambiguity (e.g., a single point on a shirt could refer to the shirt or the person wearing it).

The general framework for such a system, as shown in Fig. 1, typically consists of two main stages: a powerful visual encoder that processes the input image to create a high-dimensional feature representation, and a lightweight decoder that takes this representation along with an encoded prompt to rapidly generate the final mask.

Fig. 3. Diagram for Input/ Output here + Framework etc etc

D. Contribution

The work on the Segment Anything Model (SAM) introduced several pivotal contributions to the field. As summarized by the authors, "Our principal contributions are a new task (promptable segmentation), foundational model for that new task (SAM), and dataset (SA-1B) that make this leap possible" [8].

A key contribution is the **SA-1B dataset**, which remains the largest segmentation dataset to date. Its key features include:

- **Scale:** It contains over 11 million high-resolution images and 1.1 billion high-quality segmentation masks.
- **Privacy-First:** All images are privacy-respecting, with automated blurring of faces and license plates.
- **Diversity:** The dataset is geographically diverse, with a significant number of images from countries around the world, as illustrated in Fig. 2. This helps mitigate geographical bias often present in large-scale datasets.

The contribution of this report is to provide a comprehensive survey and structured analysis of the field of image segmentation. We synthesize the historical evolution, explain the motivations driving modern research, and offer a detailed examination of the methodology behind foundation models like SAM, contextualizing their impact and outlining future directions.

II. Related Works

A. Traditional Methods

1) K-Means Clustering: K-Means is an unsupervised clustering algorithm used to segment regions of interest from the background [10]. It was first introduced by Hartigan in 1975, and its goal is to partition M points in an N-dimensional space into K clusters such that the within-cluster sum of squares is minimized [11].

1) **Mechanics:** The algorithm iteratively assigns each pixel to the nearest cluster centroid and updates centroids based on current assignments. K-Means algorithm seeks a local optimum rather than a global one. Data points are divided into groups by assigning each of them to the nearest centroid. The goal is to find a solution such that moving any point from one cluster to another does not reduce the within-cluster sum of squares [11].
2) **Methodology:** Let $p_(x,y)$ be a pixel at position (x,y) to be cluster and c_k be the centroid of cluster k. Let C_k be the set of pixels assigned to cluster k, where (x,y) indexes the image pixels and $k \in \{1,2,\ldots,K\}$.

   For each pixel, compute its distances to all cluster centroids:

   $$D_{x,y} = \{\, \|p_{x,y} - c_k\| \mid k \in \{1,2,\ldots,K\}\}$$

   Assign the pixel to the cluster with the smallest distance:

   $$n = \arg\min_k \|p_{x,y} - c_k\|,$$
   $$C_n = C_n \cup \{p_{x,y}\}$$

   After all pixels are assigned, update each centroid as the mean of the pixels belonging to it:

   $$c_k = \frac{1}{|C_k|} \sum_{p_{x,y} \in C_k} p_{x,y}$$

   Repeat the assignment and update steps until the centroids no longer change significantly or a maximum number of iterations is reached.
3) **Input:** An image represented by M pixels, each with N features (e.g., RGB color components or combined color–spatial features), forming an $M \times N$ matrix. The algorithm also takes K initial cluster centroids in the same N-dimensional feature space.
4) **Output:**
   - A label map of size $M \times 1$ indicating the cluster each pixel belongs to.
   - A vector of size $K \times 1$ containing the number of pixels in each cluster.
   - A $K \times N$ matrix of the final cluster centroids.
5) **Algorithm:**

```
    1. Initialize cluster centroids
    2. For each pixel (x, y):
      Compute its distance to all centroids
      Assign it to the cluster with the
  smallest distance
    3. For each cluster:
      Recalculate the centroid as the mean of
   all pixels assigned to it
    4. Repeat steps 2 and 3 until convergence
```

2) Mean Shift Clustering: Mean Shift is a non-parametric clustering algorithm often used for image segmentation and object detection. Unlike K-Means, it does not require specifying the number of clusters in advance. Instead, it locates regions of high data density (modes) by iteratively shifting data points toward the nearest local maximum of a kernel density estimate [**?**].

1) **Mechanics:** The algorithm treats data points as samples from an underlying probability density function and seeks its local maxima. Each pixel is iteratively moved toward regions of higher density, determined by a kernel function, until convergence. Pixels converging to the same mode belong to the same cluster.
2) **Methodology:** Let $p_{x,y}$ denote the feature vector of the pixel at position (x,y). The goal is to move $p_{x,y}$

TABLE I
COMPARISON TABLE - PART 1: METHODOLOGY

Papers	Mechanics	Methodology		
		Stage 1	*Stage 2*	*Stage 3*
copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy copy	More table copy[a]			

[a]Sample of a Table footnote.

TABLE II
COMPARISON TABLE - PART 2: PERFORMANCE AND ANALYSIS

Papers	Performance			Contribution	Limitation
	Data	*Metric*	*Result*		
copy					

[a]Sample of a Table footnote.

iteratively toward the local density maximum according to:

$$m(p_{x,y}) = \frac{\sum_{p_j \in S} K\left(\frac{\|p_j - p_{x,y}\|^2}{h^2}\right) p_j}{\sum_{p_j \in S} K\left(\frac{\|p_j - p_{x,y}\|^2}{h^2}\right)}$$

where:

- S is the set of all data points (pixels),
- $K(\cdot)$ is a kernel function (commonly a Gaussian kernel),
- h is the bandwidth controlling the kernel width.

The mean shift vector is defined as:

$$\Delta p_{x,y} = m(p_{x,y}) - p_{x,y}$$

and the pixel is updated iteratively:

$$p_{x,y}^{(t+1)} = p_{x,y}^{(t)} + \Delta p_{x,y}$$

until convergence (i.e., when $\|\Delta p_{x,y}\| < \epsilon$ for a small threshold ϵ). After convergence, all pixels that converge to the same mode are assigned to the same cluster.

3) **Input:** An image represented by M pixels, each having an N-dimensional feature vector (e.g., color, intensity, or color–spatial components). The algorithm also requires a kernel bandwidth parameter h that defines the size of the neighborhood considered for density estimation.
4) **Output:**
   - A segmented image in which each cluster corresponds to a region of similar features.
   - The set of mode locations that represent cluster centers.
5) **Algorithm:**

```
    1. For each pixel (x, y), initialize p_(x,y) with its feature vector.
    2. Repeat until convergence:
    a. Compute the mean shift vector $\Delta$ p_(x,y)
    using nearby pixels within bandwidth h.
    b. Update p_(x,y) = p_(x,y) + $\Delta$ p_(x,y).
    3. After convergence, group pixels whose final
    positions fall within the same mode region.
    4. Assign each group as one cluster.

```

3) Binary Thresholding: Binary thresholding is used to separate the foreground (region of interest) from the background based on pixel intensity values. It is particularly effective when the objects and background have distinct intensity distributions.

1) **Mechanics:** Each pixel in the grayscale image is compared against a predefined or automatically determined threshold value T. Pixels with intensity greater than or equal to T are classified as foreground, while the others are classified as background. This process converts a grayscale image into a binary one.
2) **Methodology:** For each pixel (x, y) in the image:
   a) Compare $I(x, y)$ with the threshold value T.
   b) Assign the binary value based on the comparison rule.

   T can be predefined or determined adaptively using methods such as Otsu's algorithm, which selects the threshold that minimizes the intra-class variance between the foreground and background.
3) **Input:** A grayscale image, where (x, y) denotes the pixel coordinates, $I(x, y) \in [0, 255]$ represents its intensity, a threshold value T to determine the output pixel at (x, y).
4) **Output:** A binary image B, where

$$B(x, y) = \begin{cases} 1, & \text{if } I(x, y) \geq T, \\ 0, & \text{otherwise.} \end{cases}$$

5) **Algorithm:**

```
  For each pixel (x, y):
```

```
  If I(x, y) >= T:
    B(x, y) = 1
  Else:
    B(x, y) = 0

```

B. Modern Methods

III. Methodology

IV. Implementation & Experiment

V. Conclusion

VI. Discussion

A. Limitations

B. Future Work

References

[1] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[2] M. Kass, A. Witkin, and D. Terzopoulos, "Snakes: Active contour models," *International Journal of Computer Vision*, vol. 1, no. 4, pp. 321–331, 1988.

[3] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, vol. 1, no. 14. University of California Press, 1967, pp. 281–297.

[4] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 3431–3440.

[5] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical image computing and computer-assisted intervention*. Springer, 2015, pp. 234–241.

[6] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2961–2969.

[7] J. Chen, Y. Lu, Q. Yu, X. Luo, E. Adeli, Y. Wang, L. Lu, A. L. Yuille, and Y. Zhou, "Transunet: Transformers make strong encoders for medical image segmentation," *arXiv preprint arXiv:2102.04306*, 2021.

[8] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," *arXiv preprint arXiv:2304.02643*, 2023.

[9] N. Ravi, V. Gabeur, Y.-T. G. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson *et al.*, "SAM 2: Segment anything in images and videos," *arXiv preprint arXiv:2408.00714*, 2024.

[10] D. Nameirakpam, K. Singh, and J. Chanu, "Image segmentation using k -means clustering algorithm and subtractive clustering algorithm," *Procedia Computer Science*, vol. 54, pp. 764–771, 12 2015.

[11] J. A. Hartigan and M. A. Wong, "A k-means clustering algorithm," *Journal of the Royal Statistical Society Series C: Applied Statistics*, vol. 28, no. 1, pp. 100–108, 2018.